

HPC Tools Task 3: DGESV Profiling and Optimization

July 2026

1 Objective and Environment

The goal of Task 3 was to benchmark and analyze the custom `dgesv` implementation with Intel compilers, compare the results against the GCC campaign from Task 2, profile the code, and identify optimizations that have measurable impact.

The solver under study is the baseline Gaussian elimination with partial pivoting plus back substitution implemented in `dgesv.c`. The reference solver is LAPACK's `dgesv`; for the Intel campaign it is linked against Intel MKL.

FT3 environment used for the Intel campaign:

- `intel/2021.3.0`
- `imkl/2021.3.0`
- `icc 2021.3.0`
- `icx 2021.3.0`

The assignment requested `icx 2024.2.1`, but that version was not visible in the `cesga/2025` module tree on 2026-07-05. We therefore ran the experiments with the newest Intel toolchain that was actually available on FT3 and documented that limitation explicitly.

2 Methodology

We followed the same general methodology as in Task 2:

- matrix sizes: 1024, 2048, 4096
- three runs per configuration
- median execution time reported in the summary CSV
- run-level data stored in a separate raw CSV

New Task 3 infrastructure:

- `run_dgesv_bench_icc_2021_3_0.sh`
- `run_dgesv_bench_icx.sh`
- `profile_dgesv_intel.sh`

The Intel benchmark scripts also emit compiler vectorization reports. For profiling and analysis we used:

- `perf stat -d`
- `valgrind -leak-check=full`
- `valgrind -tool=cachegrind`

VTune and Advisor modules are available on FT3, and the helper script supports them, but the main conclusions below were already clear from `perf` and `Valgrind`.

3 Intel Benchmark Tables

3.1 ICC 2021.3.0

`fast` deserves one important note. The literal `icc -fast` driver path built a binary that crashed at runtime on FT3 with `*** stack smashing detected ***`, even after fixing the MKL link path. To keep the required `fast` table entry meaningful, we used the fast-equivalent bundle:

`-O3 -xHost -no-prec-div -fp-model fast=2 -ansi-alias`

The table label remains `fast`, but the report must mention this FT3-specific compiler-driver defect.

Matrix size	O2	O3	fast	Ref
1024x1024	2145	1868	2027	118
2048x2048	22533	22331	16745	418
4096x4096	356421	368530	194918	2774

3.2 ICX 2021.3.0

Matrix size	O2	O3	Ofast	Ofast+ipo	Ref
1024x1024	1916	1969	1915	2388	90
2048x2048	22658	22082	21012	28892	481
4096x4096	356030	369784	352261	452793	3113

4 Combined View with Task 2

The best GCC medians from Task 2 were:

- 1024: 2052 ms with GCC 8.4.0 `Ofast-vec`
- 2048: 22742 ms with GCC 8.4.0 `Ofast-vec`
- 4096: 400211 ms with GCC 8.4.0 `Ofast-vec`

The best overall medians after combining Tasks 2 and 3 are:

Relative to the best GCC result from Task 2, the Intel campaign improved the custom solver by:

- $1.10\times$ at 1024 (2052/1868)
- $1.36\times$ at 2048 (22742/16745)
- $2.05\times$ at 4096 (400211/194918)

This means the Intel toolchain advantage becomes much larger as the problem size grows.

Matrix size	Best overall configuration	Our solver (ms)	Reference (ms)
1024x1024	ICC 2021.3.0 O3	1868	88
2048x2048	ICC 2021.3.0 fast-equivalent	16745	418
4096x4096	ICC 2021.3.0 fast-equivalent	194918	2774

5 Performance Analysis

5.1 Main Findings

The most important result is that the Intel compilers, especially ICC with the fast-equivalent optimization bundle, outperform all GCC configurations from Task 2 for the large matrices.

The clearest case is 4096x4096:

- best GCC result: 400211 ms
- best ICC result: 194918 ms

That is a $2.05\times$ speedup without changing the algorithm, only the compiler configuration and runtime library stack.

5.2 Why the Intel Results Improve

The main causes are:

- better autovectorization of the row-swap and elimination loops
- more aggressive alias assumptions in the fast-equivalent ICC configuration
- stronger MKL reference performance than the OpenBLAS-based reference from Task 2

The reference solver also benefits from the Intel stack. For example, at 4096 the best GCC reference median was 4694 ms while ICC fast achieved 2774 ms, roughly $1.69\times$ faster.

5.3 Most Unexpected Results

Two results stood out as unexpected.

First, ICC fast was not the best configuration at 1024. ICC O3 achieved 1868 ms, while the fast-equivalent path took 2027 ms. For small matrices, the extra aggressiveness of the fast-equivalent flags does not compensate for setup overhead and reduced cost-model selectivity.

Second, ICX Ofast+ipo was consistently worse than plain Ofast:

- 2388 ms vs 1915 ms at 1024
- 28892 ms vs 21012 ms at 2048
- 452793 ms vs 352261 ms at 4096

This suggests that IPO increased code size or hurt locality enough that the extra whole-program optimization was not beneficial for this solver.

5.4 Quantified Optimization Impact

Within the Intel campaign, the best implemented optimization was the ICC fast-equivalent flag bundle.

Compared with ICC O2, it improved the solver by:

- $1.06\times$ at 1024
- $1.35\times$ at 2048
- $1.83\times$ at 4096

Compared with ICC O3, it improved the solver by:

- $0.92\times$ at 1024, so slightly worse
- $1.33\times$ at 2048
- $1.89\times$ at 4096

The optimization is therefore clearly beneficial for medium and large problems, which are the more relevant HPC cases.

6 Vectorization Analysis

6.1 Are the Key Parts Autovectorized?

Yes. The key numeric loops in `dgesv.c` are autovectorized by both Intel compilers.

From the ICX Ofast report:

- row swap loop over `a` is vectorized
- row swap loop over `b` is vectorized
- elimination update over `a` is vectorized
- elimination update over `b` is vectorized

From the Intel optimization report already present in the repository (`ipo_out.opttrpt`), ICC also vectorizes:

- pivot search inner loop at `dgesv.c(8,5)`
- row swap loops at `dgesv.c(17,7)` and `dgesv.c(23,7)`
- elimination loops at `dgesv.c(34,7)` and `dgesv.c(38,7)`
- one back-substitution inner loop at `dgesv.c(47,7)`

The ICC report even estimates a potential speedup of about $4.22\times$ for the elimination update loops after vectorization.

6.2 What Is Not Vectorized, and Why?

Some loops still resist vectorization:

- `generate_matrix()` is not vectorized because it calls `rand()`
- `check_result()` is not vectorized in the ICC fast report because it uses an unsigned induction variable and has early exit behavior
- the outer pivot-search loop is not vectorized by ICX because the reduction and selected row index escape the loop
- parts of back substitution remain limited by loop-carried dependences and by strided access to `b[j*nrhs + rhs]`

This behavior is consistent with the algorithm. The row-wise loops are friendly to SIMD, but the dependency structure in pivoting and back substitution is inherently harder.

6.3 Can Source Changes Help?

Yes, but the benefit is limited because the baseline already vectorizes the main row-based loops under Intel compilers.

Possible source-level improvements are:

- replace unsigned loop counters in non-critical helper loops with signed integers
- separate pivot search from row index updates more clearly to help ICX identify the reduction
- use `restrict`-based helper routines similar to the separate autovectorization experiment from Task 2
- improve alignment guarantees for matrix allocations

However, for the Task 3 baseline, compiler-flag tuning delivered larger gains than additional source refactoring.

7 Memory and Cache Behaviour

7.1 Heap Usage

The first Valgrind pass showed that `main.c` leaked the five main allocations:

- `a`
- `b`
- `aref`
- `bref`
- `ipiv`

We fixed that by freeing all owned buffers before returning from `main()`. After the fix, Valgrind reports:

- definitely lost: 0 bytes

- indirectly lost: 0 bytes
- possibly lost: 0 bytes

There are still some reachable blocks inside MKL and loader internals, but the program's own allocations are no longer leaked.

7.2 Access Pattern

The solver uses row-major matrices. That creates two very different memory behaviors.

Good spatial locality appears in:

- row swaps over `a[i*n + j]`
- row swaps over `b[i*nrhs + rhs]`
- elimination updates over `a[k*n + j]`
- elimination updates over `b[k*nrhs + rhs]`

These loops walk contiguous row segments and are exactly the loops that the compilers vectorize most effectively.

Poorer locality appears in:

- pivot search, which reads `a[k*n + i]` and therefore traverses a column in row-major storage
- back substitution, which accesses `b[j*nrhs + rhs]` and is also strided in the inner loop

These accesses have weaker spatial locality, create more cache pressure, and are harder to vectorize efficiently.

7.3 Cache Observations

The profiling results support the idea that the solver becomes memory-sensitive as the matrices grow.

`perf stat -d` on ICX Ofast at size 512 showed:

- IPC around 1.15
- L1 data-cache miss rate around 33.5%
- LLC-load miss rate around 99.17% of LLC loads

This is consistent with a kernel that streams large working sets and frequently fetches data from deeper cache levels.

`cachegrind` on the optimized ICC fast-equivalent path at size 256 showed:

- D1 miss rate around 5.4%
- LL miss rate around 1.1%

Those smaller-size results are qualitatively useful even if Valgrind is not appropriate for absolute timing. They show that cache behavior is still reasonable for smaller matrices, while the hardware `perf` counters indicate much heavier pressure once the matrix grows.

8 Profiling Summary

The profiling tools used in practice were:

- `perf stat -d`
- `valgrind -leak-check=full`
- `valgrind -tool=cachegrind`

Main conclusions from profiling:

- the solver is not branch-miss dominated; branch-miss rates stayed low in the `perf` runs
- the key bottleneck is memory traffic in the large row-update phases and the strided accesses in pivot search and back substitution
- the custom solver allocates the correct matrix sizes, and after the memory fix it no longer leaks its own heap allocations

9 Implemented Optimizations

The main implemented optimizations in Task 3 were:

1. Intel benchmark infrastructure.

We added dedicated scripts for ICC and ICX, with median-of-three reporting and vectorization-report emission.

2. ICC fast workaround.

Because the literal `icc -fast` path crashed at runtime on FT3, we replaced it with the fast-equivalent flag bundle:

```
-O3 -xHost -no-prec-div -fp-model fast=2 -ansi-alias
```

This preserved the intended aggressive optimization behavior while producing stable binaries and the best large-matrix results.

3. Memory-management fix.

We added missing `free()` calls in `main.c`, eliminating the definite leaks reported by Valgrind.

4. Reusable profiling helper.

We added `profile_dgesv_intel.sh` so that `perf`, `cachegrind`, `massif`, `advisor`, and `vtune` can be run consistently on the Intel binaries.

5. Separate optimized experiment.

We created a separate experimental solver in `dgesv_intel_opt.c` so that the baseline and the earlier helper-based variant remain untouched. This version combines three ideas:

- `restrict`-qualified row pointers
- manually unrolled contiguous row helpers for swaps, updates, and scaling
- a rewritten back-substitution phase that operates on complete RHS rows instead of strided scalar updates

Size	Older helper-based variant (ms)	<code>dgesv_intel_opt.c</code> (ms)	Speedup
512	176	81	2.17x
1024	1731	641	2.70x
2048	16234	6663	2.44x
4096	194615	62337	3.12x

Table 1: Separate optimized experiment under ICC 2021.3.0 fast-equivalent flags.

The complete single-run comparison under the same ICC fast-equivalent flags used in the main Task 3 campaign is shown in Table 1.

The dedicated artifact for this comparison is stored in `dgesv_results_icc_separate_opt.csv`.

These results strengthen the earlier conclusion: back substitution was a larger bottleneck than the row-update kernels alone, and reorganizing the solve to operate on contiguous RHS rows is substantially more effective than only adding helper-based swap and update routines. The benefit also scales well with problem size, exceeding 3x at 4096.

10 Final Conclusion

The Task 3 results show that compiler and runtime choice matter a lot for this solver. The Intel toolchain produced the best results of the whole assignment campaign, especially for the larger matrices where the gap over GCC became large.

The best overall configuration in this repository is:

- ICC 2021.3.0 fast-equivalent for 2048 and 4096
- ICC 2021.3.0 O3 for 1024

The main reasons are:

- successful autovectorization of the row-based hot loops
- aggressive but still stable floating-point and aliasing optimizations in the ICC fast-equivalent path
- stronger MKL performance for the reference solver

The code is still fundamentally limited by the algorithm’s memory behavior: column-like pivot search and strided back substitution remain difficult for caches and SIMD. That is why profiling points to memory pressure rather than control-flow overhead as the main scaling problem.

If more time were available, the next experiments we would prioritize are:

- running the same campaign on a true `icx 2024.2.1` installation
- benchmarking the `dgesv_autovec.c` helper-based variant under Intel compilers
- testing aligned allocation and explicit SIMD hints on the pivot and back-substitution kernels